



陈宇航 按时通关

学号：U202315752

分班：23本硕博

截止前完成关卡：4/4

最新完成关卡：4/4

完成效率：--

课堂最高完成效率：--

| 通关时间             | 计时规则 ? | 实训总耗时 ?    | 评测次数 | 查重扣分 | 补交扣分 | 最终成绩      | 总评 |
|------------------|--------|------------|------|------|------|-----------|----|
| 2025-05-11 15:14 | 页面停留时长 | 2时 19分 13秒 | 13   | --   | --   | 24.0/24.0 | 优秀 |

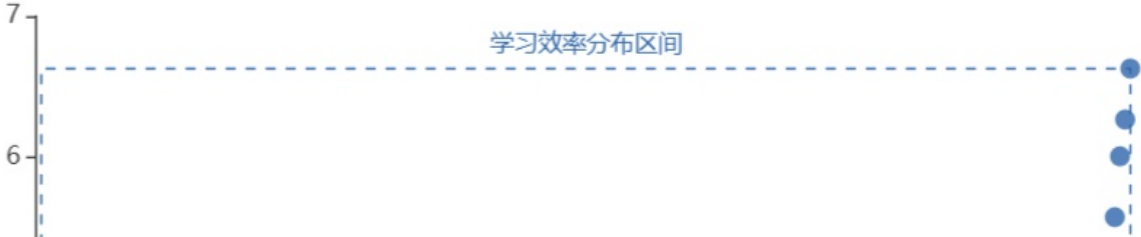
阶段成绩

| 关卡 | 任务名称                        | 开启时间             | 代码修改行数 | 评测次数 | 完成时间             | 实训耗时     | 是否查看答案 | 经验值       | 关卡得分   | 调分   |
|----|-----------------------------|------------------|--------|------|------------------|----------|--------|-----------|--------|------|
| 1  | 磁盘存储管理器(Disk Manager)       | 2025-04-29 23:42 | 69     | 6    | 2025-05-09 11:39 | 1时 9分 6秒 | 否      | 1000/1000 | 6.00/6 | 6.00 |
| 2  | 缓冲池替换策略(Page Replacer)      | 2025-05-09 15:21 | 24     | 5    | 2025-05-09 16:21 | 27分 33秒  | 否      | 1000/1000 | 6.00/6 | 6.00 |
| 3  | 缓冲池管理器(Buffer Pool Manager) | 2025-05-09 16:21 | 82     | 1    | 2025-05-11 10:33 | 12分 44秒  | 否      | 1000/1000 | 6.00/6 | 6.00 |
| 4  | 记录管理器(Record Manager)       | 2025-05-11 10:34 | --     | 1    | 2025-05-11 15:14 | 29分 50秒  | 否      | 700/700   | 6.00/6 | 6.00 |

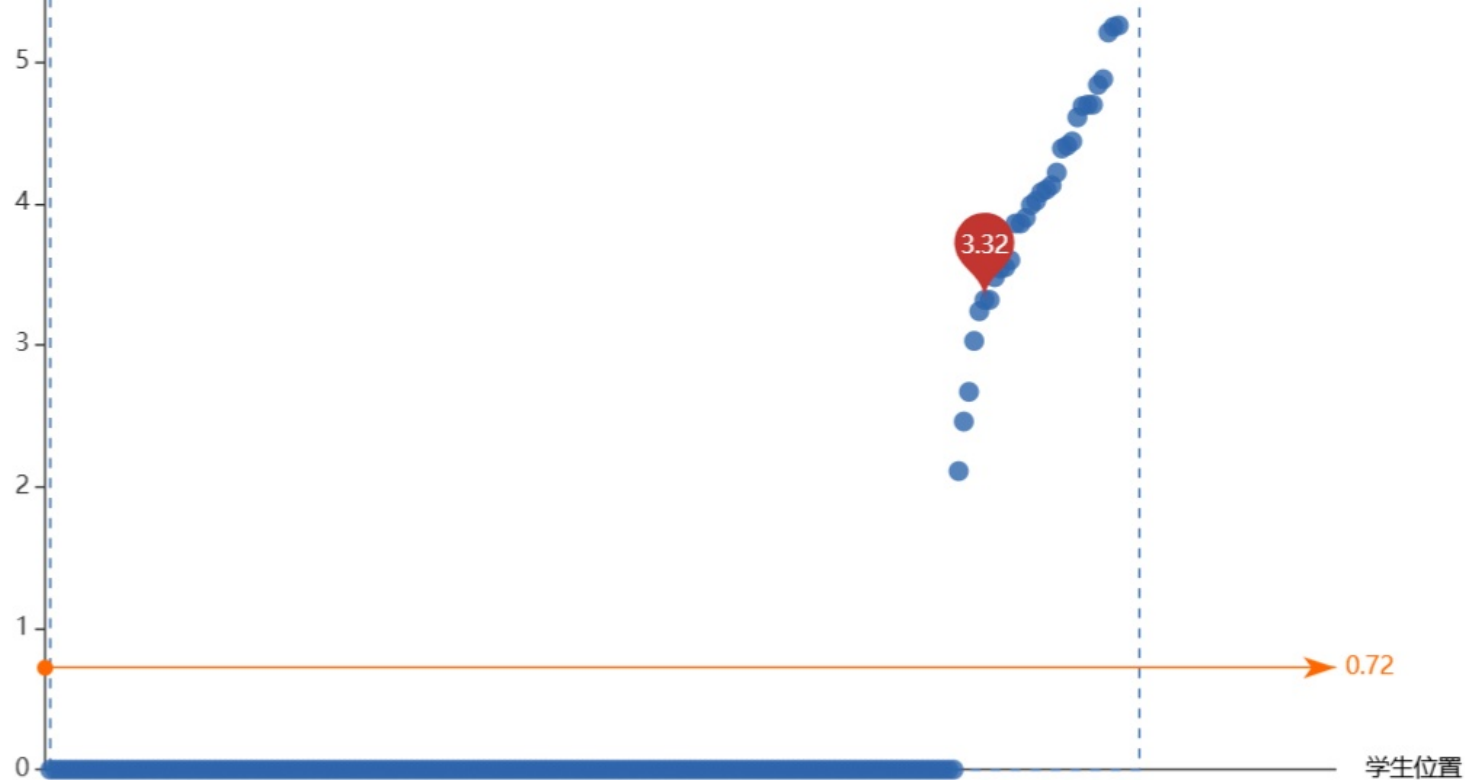
图形统计

学习效率

学习效率=log(实训总得分/实训总耗时)



姓名 陈宇航  
学号 U202315752  
学习效率 3.32  
当前排名 30



1个小圆点代表1个学生的学习效率  
红色水滴表示当前学生的学习效率值  
小圆点越高，学习效率越高  
橙色横向箭头：中位值，表示整个课堂学生表现的平均值

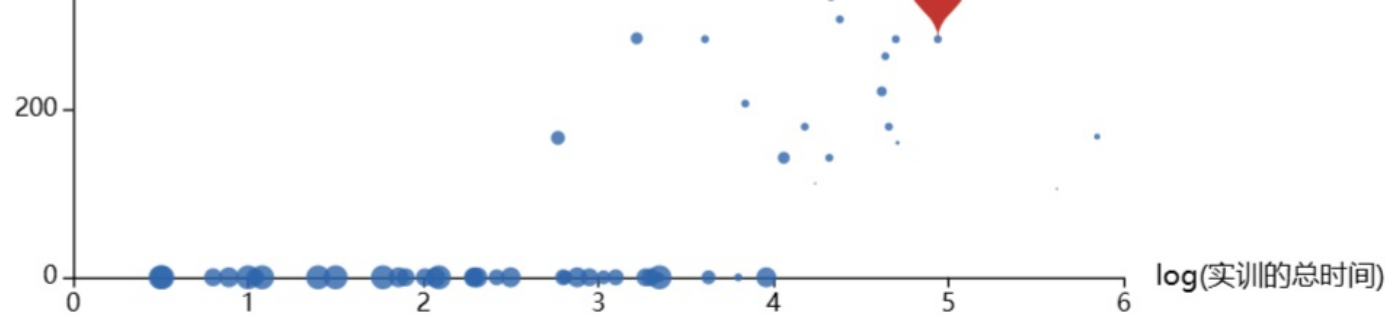
## 能力值

能力值 (实训获得经验值/实训评测次数)



1个小圆点代表1个学生的能力值  
红色水滴表示当前学生的能力值  
小圆圈越大，评测次数越少

|    |            |
|----|------------|
| 姓名 | 陈宇航        |
| 学号 | U202315752 |
| 能力 | 284.62     |



实训详情



学员代码

代码修改行数 69

rmdb/src/storage/disk\_man

复制代码

```
1  #include "storage/disk_manager.h"
2
3  #include <assert.h>    // for assert
4  #include <string.h>    // for memset
5  #include <sys/stat.h>  // for stat
6  #include <unistd.h>    // for lseek
7
8  #include "defs.h"
9
10 DiskManager::DiskManager() {
11     memset(fd2pageno_, 0, MAX_FD * (sizeof(std::atomic<page_id_t>) / sizeof(char)));
12 }
13
14 /**
15  * @brief Write the contents of the specified page into disk file
16  *
```

| 评测次数 | 详细信息                 | 评测耗时(S) | 内存消耗(MB) | 评测时间             | 操作                   |
|------|----------------------|---------|----------|------------------|----------------------|
| 1    | 共有1组测试集，其中有1组测试结果不匹配 | 23.49   | 232.99   | 2025-05-09 11:31 | <a href="#">查看代码</a> |
| 2    | 共有1组测试集，其中有1组测试结果不匹配 | 2.04    | 189.3    | 2025-05-09 11:36 | <a href="#">查看代码</a> |
| 3    | 评测通过                 | 133.31  | 706.82   | 2025-05-09 11:39 | <a href="#">查看代码</a> |

代码评价

暂无评价~



学员代码

代码修改行数 24

rmdb/src/replacer/lru\_replac

复制代码

```
1 #include "lru_replacer.h"
2
3 LRUReplacer::LRUReplacer(size_t num_pages) { max_size_ = num_pages; }
4
5 LRUReplacer::~~LRUReplacer() = default;
6
7 /**
8  * @brief 使用LRU策略删除一个victim frame, 这个函数能得到frame_id
9  * @param[out] frame_id id of frame that was removed, nullptr if no victim was found
10  * @return true if a victim frame was found, false otherwise
11  */
12 bool LRUReplacer::Victim(frame_id_t *frame_id) {
13     // C++17 std::scoped_lock
14     // 它能够避免死锁发生, 其构造函数能够自动进行上锁操作, 析构函数会对互斥量进行解锁操作, 保证线程安全。
15     std::scoped_lock lock{latch_};
16 }
```

| 评测次数 | 详细信息                  | 评测耗时(S) | 内存消耗(MB) | 评测时间             | 操作                   |
|------|-----------------------|---------|----------|------------------|----------------------|
| 1    | 共有1组测试集, 其中有1组测试结果不匹配 | 1.97    | 137.77   | 2025-05-09 16:13 | <a href="#">查看代码</a> |
| 2    | 共有1组测试集, 其中有1组测试结果不匹配 | 0.73    | 140.12   | 2025-05-09 16:14 | <a href="#">查看代码</a> |
| 3    | 共有1组测试集, 其中有1组测试结果不匹配 | 0.76    | 141.29   | 2025-05-09 16:17 | <a href="#">查看代码</a> |
| 4    | 共有1组测试集, 其中有1组测试结果不匹配 | 1.44    | 141.29   | 2025-05-09 16:20 | <a href="#">查看代码</a> |
| 5    | 评测通过                  | 31.36   | 331.76   | 2025-05-09 16:21 | <a href="#">查看代码</a> |

代码评价

暂无评价~



学员代码

代码修改行数 82

rmdb/src/storage/buffer\_poc

复制代码

```
1  #include "buffer_pool_manager.h"
2
3  /**
4   * @brief 从free_list或replacer中得到可淘汰帧页的 *frame_id
5   * @param frame_id 帧页id指针,返回成功找到的可替换帧id
6   * @return true: 可替换帧查找成功 , false: 可替换帧查找失败
7   */
8  bool BufferPoolManager::FindVictimPage(frame_id_t *frame_id) {
9      // Todo:
10     // 1 使用BufferPoolManager::free_list_判断缓冲池是否已满需要淘汰页面
11     // 1.1 未满获得frame
12     // 1.2 已满使用lru_replacer中的方法选择淘汰页面
13
14     if (this->free_list_.empty()) {
15         if (!this->replacer_->Victim(frame_id)) { // 空闲帧不足,调用LRU淘汰
16             return false; // 淘汰失败
```

| 评测次数 | 详细信息 | 评测耗时(S) | 内存消耗(MB) | 评测时间             | 操作                   |
|------|------|---------|----------|------------------|----------------------|
| 1    | 评测通过 | 39.51   | 544.01   | 2025-05-11 10:33 | <a href="#">查看代码</a> |

代码评价

暂无评价~



学员代码

rmdb/src/record/rm\_file\_han

复制代码

```
1  #include "rm_file_handle.h"
2
3  /**
4   * @brief 由Rid得到指向RmRecord的指针
5   *
6   * @param rid 指定记录所在的位置
7   * @return std::unique_ptr<RmRecord>
8   */
9  std::unique_ptr<RmRecord> RmFileHandle::get_record(const Rid &rid, Context *context) const {
10     // Todo:
11     // 1. 获取指定记录所在的page handle
12     // 2. 初始化一个指向RmRecord的指针（赋值其内部的数据和size）
13     RmPageHandle tmp_page_handle = fetch_page_handle(rid.page_no);
14     int size_ = tmp_page_handle.file_hdr->record_size;
15     char *data_ = tmp_page_handle.get_slot(rid.slot_no);
16     //printf("在get record里面为%s\n",data );
```

| 评测次数 | 详细信息 | 评测耗时(S) | 内存消耗(MB) | 评测时间             | 操作                   |
|------|------|---------|----------|------------------|----------------------|
| 1    | 评测通过 | 9.97    | 329.27   | 2025-05-11 15:14 | <a href="#">查看代码</a> |

## 代码评价

暂无评价~